



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: M.C Rene Adrian Davila Perez

Asignatura: Programación orientada a objetos

Grupo: 7

No de Práctica(s): 4

Integrante(s): 323036285
323279866
323209432

323161891
323063609
314306908

No. De Lista o brigada 3

Semestre: 2027-1

Fecha de entrega: 25/ 09 / 2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	3
3. Desarrollo	5
4. Resultados	12
5. Conclusiones	14

1. Introducción

Actualmente los estudiantes de diferentes áreas usan los modelos de lenguaje a gran escala para consultar conceptos técnicos específicos de programación, lo que genera un gran problema ya que muchos estudiantes usan la información sin verificarla y sin saber si es correcta o no, ya que estos modelos pueden confundir o mezclar conceptos entre distintos lenguajes de programación, afectando la calidad y confianza del aprendizaje.

Al mismo tiempo, al iniciar en un nuevo lenguaje de programación al estudiantado se le presentan problemas que tienen múltiples soluciones, lo cuál se vuelve un problema a la hora de programar, ya que si no se manejan correctamente las estructuras más simples del lenguaje, un programa muy simple puede terminar siendo mucho más complejo de lo que se esperaba.

Resolver estos problemas nos ayudará a usar mejor a los LLM a la hora de consultar conceptos técnicos, consiguiendo una mejor comprensión y más veracidad en nuestro aprendizaje, también nos ayudará a comprender mejor el lenguaje de programación Java, y a practicar para encontrar las soluciones más simples y óptimas.

Objetivos

- Investigar con apoyo de nuestro LLM el concepto de constructor en el lenguaje de Java empleando un prompt eficiente para evitar que nos dé información inútil.
- Elaborar un programa que determine el número de palabras duplicadas en una oración utilizando los métodos y estructuras adecuadas, para lograr un entendimiento correcto del tema y desarrollar nuestras habilidades de programación.

2. Marco Teórico

En esta práctica se requirió el uso de estructuras de datos, utilizando `HashMap` como forma de contar la frecuencia en la que las palabras se repiten [2]. Al manejar cadenas de texto en Java se necesitaron de funciones que nos permitieran manipularlas:

- `toLowerCase()`: Normaliza todo el texto de la cadena y lo escribe como minúsculas.
- `replaceAll(regex, )`: Permite eliminar todos aquellos caracteres que no sean deseados con sus variantes regulares, con `regex` siendo el patrón que debe corroborar para aplicar estos cambios.
- `isEmpty()`: Valida si la cadena se encuentra vacía, se comporta como booleano.
- `trim()`: Elimina los caracteres en blanco de una cadena en sus extremos.

A su vez se requirió la siguiente estructura de datos para listar la cuenta de las palabras que han sido ordenadas de forma alfabética:

`TreeSet` es una colección que almacena sus elementos de forma organizada y sin registrarlos de forma duplicada, forma parte de `java.util.package` [1].

Funciones utilizadas en `HashMap` y `TreeSet`

Para el manejo de los datos ingresados en los `HashMap` se necesitan las siguientes funciones:

- `containsKey()`: Se utiliza para comprobar si una llave existe en un `TreeMap` [3].
- `get()`: Obtiene el elemento pedido en base a una llave.

- `put(llave, valor)`: Si se necesita ingresar un elemento al mapa se usa esta función; en caso de ya existir un elemento con la misma llave se reemplazará.
- `keySet()`: Regresa el conjunto de llaves que se encuentran en el mapa.
- `add()`: Nos permite ingresar los valores al `TreeSet`.

3. Desarrollo

Para este punto de la práctica se le preguntó a un LLM acerca de los destructores, para leer y analizar detenidamente lo que decía, a continuación se presenta su respuesta:

1. El concepto: ¿Java tiene destructores?

En POO, un destructor es un método especial que se ejecuta automáticamente cuando un objeto va a dejar de existir, para liberar recursos (memoria, archivos abiertos, conexiones, etc.). En C++ existen destructores explícitos.

En Java, NO hay destructores como tal. Esto es la primera cosa importante que debes entender. Java tiene un recolector de basura (Garbage Collector, GC) que se encarga automáticamente de liberar la memoria de los objetos que ya no se usan. Tú no decides cuándo se destruye un objeto ni cómo.

2. ¿Entonces qué usa Java en su lugar?

Existen dos mecanismos relacionados:

a) `finalize()` (obsoleto, ya no se debe usar).

Antes existía un método llamado `finalize()` que el GC podía llamar antes de destruir un objeto. Está deprecado (desde Java 9) y eliminado en versiones recientes porque es poco confiable.

b) `AutoCloseable` / método `close()` (la forma moderna y correcta).

Si un objeto usa un recurso externo (archivo, conexión a base de datos, socket), Java usa la interfaz `AutoCloseable`, que obliga a implementar un método `close()`. Este se llama de forma explícita o automática con `try-with-resources`.

Ejemplo con `finalize()`

```
[language=Java] public class Recurso { public Recurso() { System.out.println("Recurso creado");
```

```
@Override protected void finalize() throws Throwable System.out.println("El
recurso está siendo destruido por el GC");
public static void main(String[] args) Recurso r = new Recurso(); r =
null; System.gc();
```

Aquí `finalize()` funciona como un "destructor", pero el problema es que no tienes control de cuándo se ejecuta.

Ejemplo con AutoCloseable

```
[language=Java] public class Recurso implements AutoCloseable public
Recurso() System.out.println("Recurso abierto");
public void usar() System.out.println("Usando el recurso...");
@Override public void close() System.out.println("Recurso cerrado (libe-
rado)");
public static void main(String[] args) try (Recurso r = new Recurso())
r.usar();
```

Salida:

Recurso abierto

Usando el recurso...

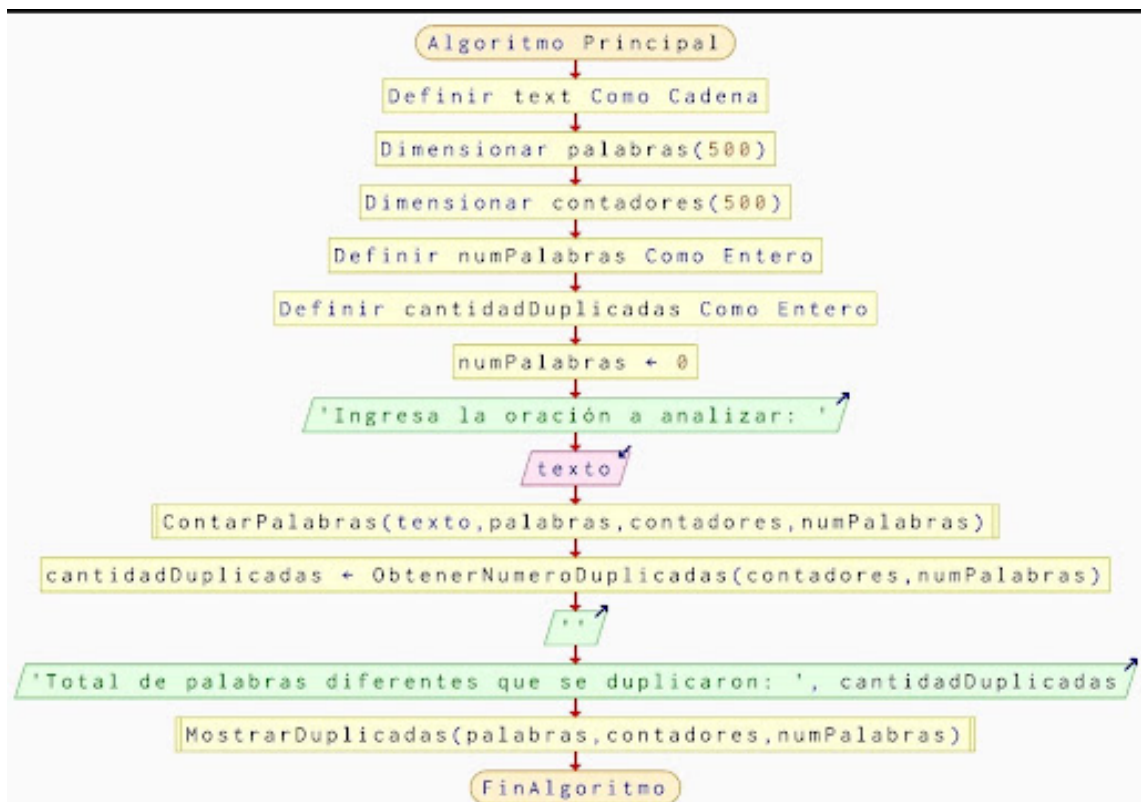
Recurso cerrado (liberado)

Resumen

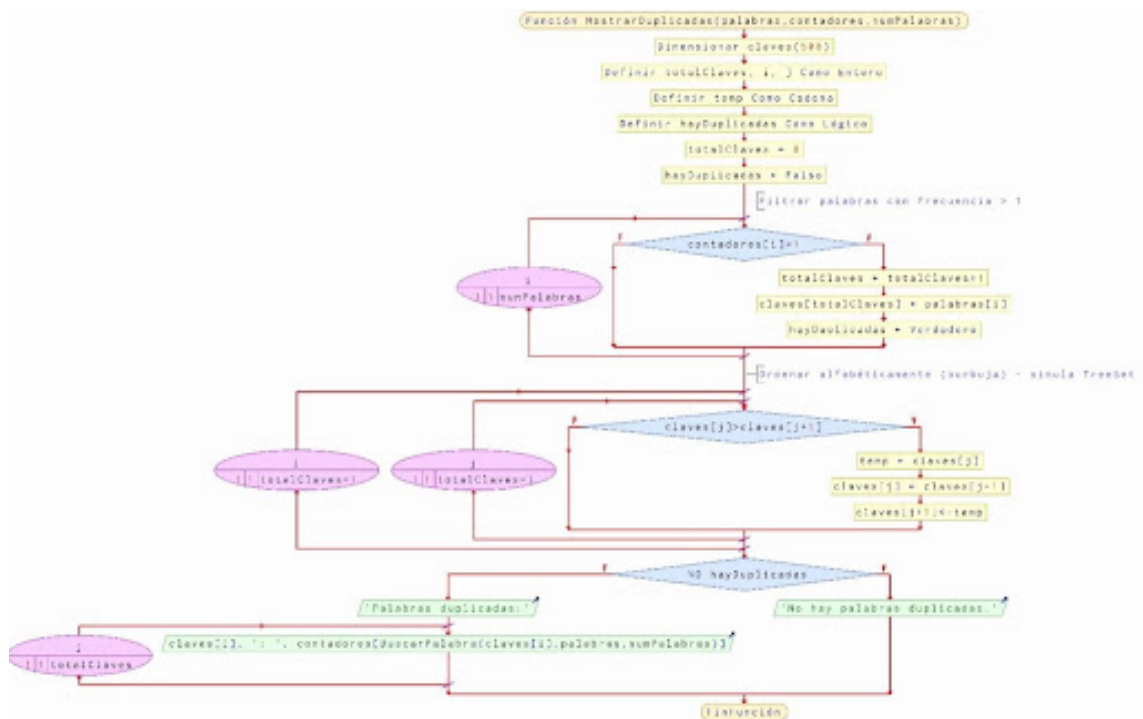
Concepto	¿Existe en Java?	Uso recomendado
Destructor explícito (como C++)	No	—
<code>finalize()</code>	Sí, pero obsoleto	No usarlo
<code>AutoCloseable + close()</code>	Sí, moderno	Usarlo con <code>try-with-resources</code>
Garbage Collector	Sí, automático	Libera memoria, no se controla

Ejercicio: Analizador de Palabras

Para el segundo ejercicio se emplearon clases, métodos y estructuras de datos abstractas, para que así se pudiera trabajar con las palabras, verificar cuáles están repetidas y cuantas veces se han repetido, se siguió el siguiente diagrama:



La primera clase se llamó `AnalizadorPalabras` y en ella se fueron usaron diferentes métodos. Para ésta clase se importaron desde `java.util` las interfaces `Collection` y `Map`, y las clases `HashMap`, la cuál implementa a la interfaz `Map`, y `TreeSet`.



Para iniciar, se declaró un atributo **oracion** de tipo **String**, donde se almacenará la oración introducida por el usuario.. Después se declaró un atributo **frecuencia**, usando el tipo general **Map** fabricandolo como un objeto de la clase **HashMap**, para relacionar un tipo de dato **String** con un tipo de dato **Integer**.

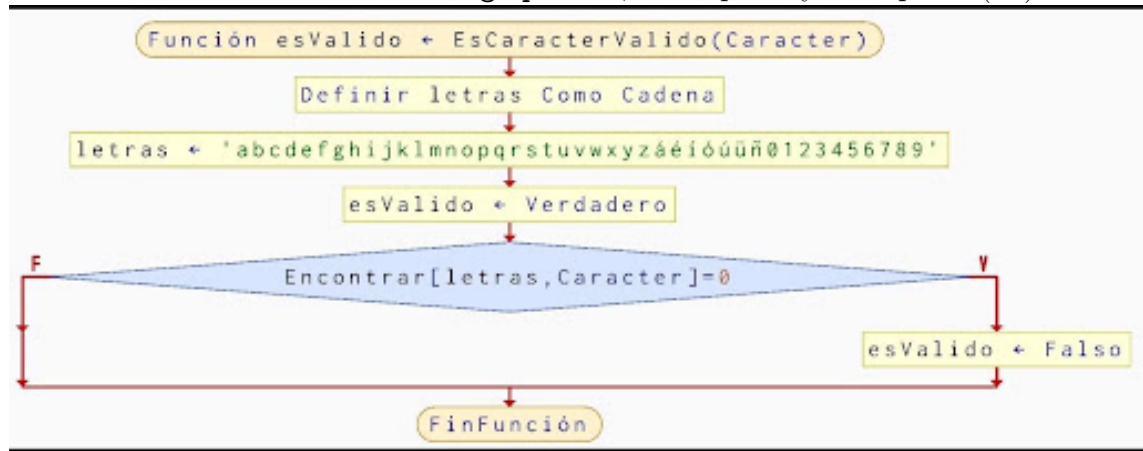
Posteriormente se definió el constructor de la clase **AnalizadorPalabras**, el cuál recibe como parámetro la cadena de texto a analizar, la asigna al atributo **oración** y crea en la memoria RAM un tipo de dato **HashMap**.

Después de esto, se añadieron los métodos encargados del procesamiento y análisis de texto:

El método **contarPalabras**, es donde empieza el análisis de las palabras. Aquí se comienza verificando que la oración contenga algo; si el atributo no apunta a nada en la memoria o en la verificación de **trim**, método de la clase **String** que elimina los espacios al principio y al final de la oración, se ve que la oración está en blanco, se interrumpe la ejecución del método. Si la oración contiene algo, se procede a “limpiarla”, pasando las palabras a

minúsculas con el método `toLowerCase()` y remplazando caracteres extraños con `replaceAll()`, ambos métodos de la clase `String`.

Después de limpiar, se declara una variable `tokens`, que es un arreglo de cadenas de texto, y los datos que se le añaden son las palabras separadas con el método nativo de la clase `String split()`, cada que hay un espacio (`s+`).



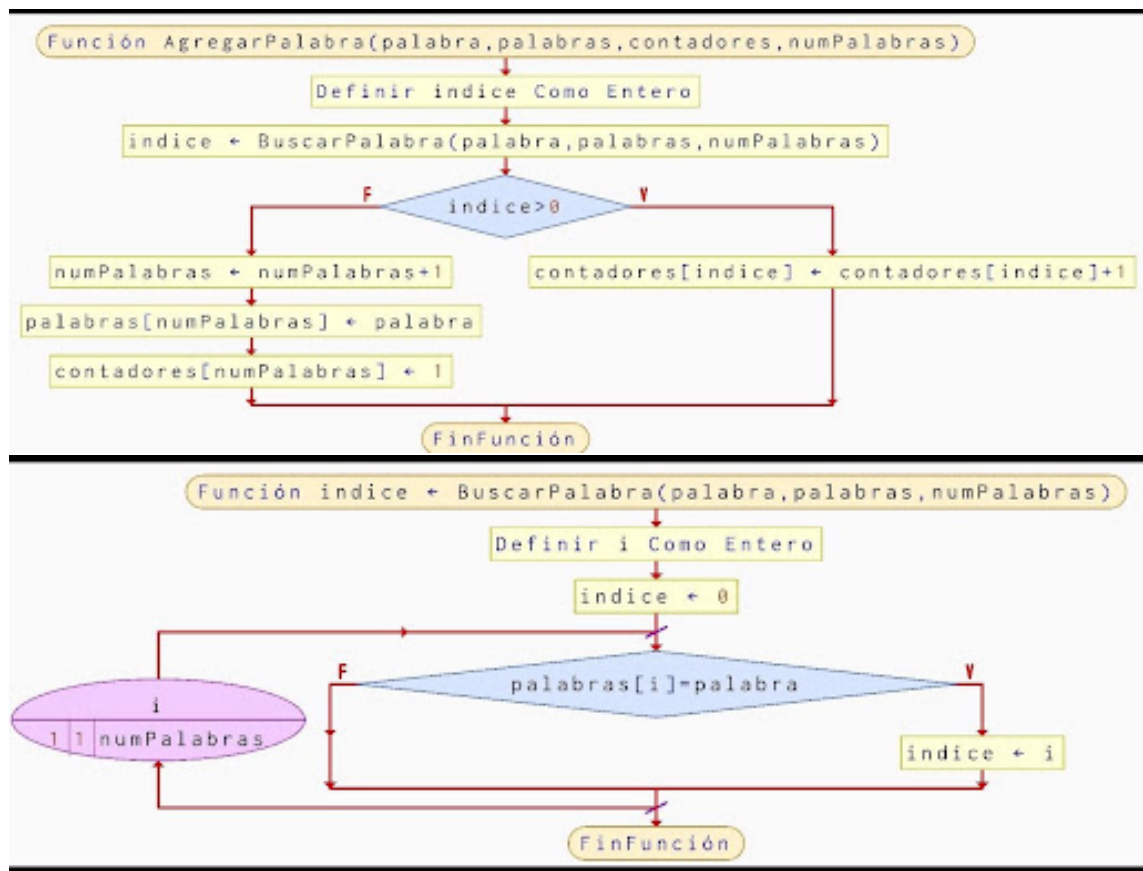
Para el conteo se usa un bucle `for-each`, para recorrer el arreglo `tokens` cada que se corte con `split`, y se guarda en la variable local `palabra`.

Dentro del bucle se filtran los elementos, para evitar registrar espacios fantasma que hayan quedado tras limpiar la oración.

Posteriormente se consulta el mapa con el método de `HashMap` `containsKey()` y se revisa que la palabra actual ya exista como llave en el atributo `frecuencia`.

Posteriormente se emplea el método `get()`, también de la clase `HashMap`, para que si la palabra ya existía busque cuántas veces se había registrado antes, y guarde la cifra en la variable `contador`. Con el método `put()` se sobrescribe la llave con el nuevo conteo incrementado en 1 el contador.

Finalmente, si la palabra no existía en el mapa se le inserta por primera vez asociando el número 1 como frecuencia inicial.



Para el método `obtenerNumeroDuplicadas` se declara una variable local `duplicados` con tipo de dato `int`, y se inicializa en 0.

Con un bucle `for-each` se recorre la colección de palabras obtenida con el método `keySet()` de `HashMap` y se le asigna a la variable local `llave`.

Con el método `get()` se verifica que la frecuencia sea mayor a 1, y si es así la variable `duplicados` es aumentada en 1.

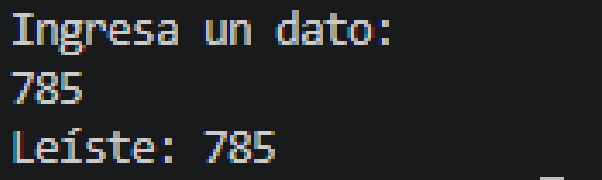
4. Resultados

Al correr en la consola el `GarbageCollector`, este fue el resultado que nos arrojó:

```
PS C:\Users\Angelica> & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe'  
'-agentlib:jdpw=transport=dt_socket,server=n,suspend=y,address=localhost:50816'  
'--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages'  
'-cp' 'C:\Users\Angelica\AppData\Local\Temp\vscodesws_65944\jdt_ws\jdt.ls-java-  
project\bin'  
'GarbageCollector'
```

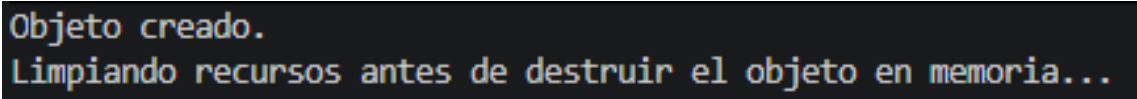
```
PS C:\Users\Angelica>
```

Al alcanzar el cierre del mecanismo try-with-resources se cerró el flujo de entrada de datos y se libero la memoria ocupada por el objeto sc:



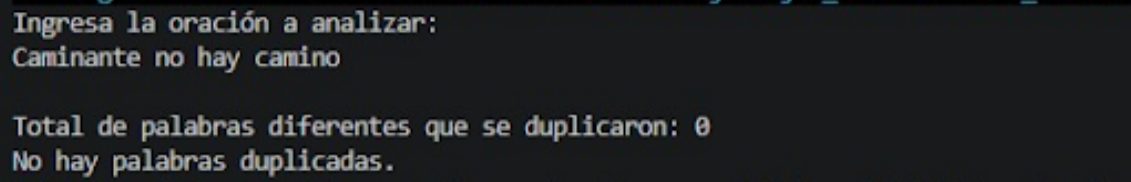
```
Ingresa un dato:  
785  
Leíste: 785
```

Gracias a la instrucción `System.gc()` se ejecuto el recolector de basura. Tras detectar que el objeto quedó huérfano, la máquina virtual de java se borro de la memoria. Sin embargo, VisualStudio nos recuerda que este método es obsoleto.



```
Objeto creado.  
Limpiando recursos antes de destruir el objeto en memoria...
```

Al iniciar el programa el usuario ingresará n número de palabras y el código nos indicará cuando una palabra se repite, cuál lo hace y cuantas veces.



```
Ingresa la oración a analizar:  
Caminante no hay camino  
  
Total de palabras diferentes que se duplicaron: 0  
No hay palabras duplicadas.
```

En este caso no existen palabras repetidas; el programa lo reconoce y por lo tanto, no hay conteo de cuantas repeticiones hay.

```
Ingresar la oración a analizar:  
Hola hola Hóla HOLá HOla  
  
Total de palabras diferentes que se duplicaron: 1  
Palabras duplicadas:  
hola: 3
```

En este ejemplo vemos que el programa hace distinciones entre mayúsculas y minúsculas, más no con acentos, por eso la palabra hola la cuenta repetida 3 veces y no 5.

```
Ingresar la oración a analizar:  
You say, "Goodbye", and I say, "HelloHello, hello"I don't know why you say, "Goodbye", I say, "HelloHello, hello"I don't know why you say,  
"Goodbye", I say "Hello"  
  
Total de palabras diferentes que se duplicaron: 9  
Palabras duplicadas:  
dont: 2  
goodbye: 3  
hellohello: 2  
helloi: 2  
i: 3  
know: 2  
say: 6  
why: 2  
you: 3
```

Por último, podemos constatar que el programa hace una distinción caso de haber comas o comillas y que es capaz de contabilizar más de una palabra repetida.

5. Conclusiones

Se cumplieron los objetivos de esta práctica. En primer lugar, investigamos con ayuda de nuestros LLM's el concepto de constructor en Java y lo aplicamos en el programa, lo que nos ayudó a entender que un constructor sirve para dejar al objeto en un estado válido desde su creación, no solo para crearlo.

En segundo lugar, logramos elaborar un programa funcional que determina las palabras duplicadas en una oración usando las estructuras: `contarPalabras()`, que limpia y normaliza el texto para después contar cada palabra con ayuda de `containsKey()`, `get()` y `put()` sobre un `HashMap`, mientras que `obtenerNumeroDuplicadas()` y `mostrarDuplicadas()` identifican y muestran esas repeticiones ya ordenadas alfabéticamente gracias a un `TreeSet`. Las pruebas confirmaron que el programa distingue correctamente mayúsculas de minúsculas, ignora signos de puntuación y maneja bien oraciones largas, por lo que se completaron de manera satisfactoria los ejercicios que se dejaron.

Referencias

- [1] GeeksforGeeks. *Hashing in Java*. <https://www.geeksforgeeks.org/java/ hashing-in-java/>. Publicado: 11 de julio de 2025.
- [2] GeeksforGeeks. *HashMap in Java*. <https://www.geeksforgeeks.org/java/java-util-hashmap-in-java-with-examples/>. Publicado: 31 de agosto de 2026.
- [3] GeeksforGeeks. *Java TreeMap containsKey() method*. <https://www.geeksforgeeks.org/java/java-treemap-containskey-method/>. Publicado: 11 de julio de 2025.